DOCKER: WHEN YOUR COMPILER IS TOO OLD

A modern GCC and Clang, without touching your system

WHY DOCKER

The problem it solves

Your system's GCC/Clang (or an older Kit in Qt Creator) might not support everything this course uses, and upgrading it isn't always simple - or you might not want to risk changing what's already installed and working for other projects.

Docker gives you a **container**: an isolated Linux environment with its own compiler, completely separate from your host machine.

WHAT THIS COURSE PROVIDES

Two ready-made images

IMAGE	COMPILER	BASE
`masterclass-gcc:16`	GCC 16.1.0	Debian 13
`masterclass-clang:21`	Clang 21.1.8	Debian 13

Both include CMake, Ninja, git, a debugger (gdb/lldb), clang-format, clang-tidy, cppcheck, Catch2, GoogleTest, and vcpkg - everything this course's lecture folders need, pre-installed.

BUILDING THE IMAGES

One-time setup

From the repo root:

```
docker build -t masterclass-gcc:16 docker/gcc
docker build -t masterclass-clang:21 docker/clang
```

You only do this once (or again if the Dockerfiles change).

Mount a lecture folder, build inside it

```
# Windows (PowerShell)
docker run --rm -it -v "${PWD}\02.EnvSetup\2.2MeetTheToolchain:/workspace" masterclass-gcc:16
```

```
# Linux / macOS
docker run --rm -it -v "$(pwd)/02.EnvSetup/2.2MeetTheToolchain:/workspace" masterclass-gcc:16
```

Then, inside the container:

```
cmake -B build -G Ninja .
cmake --build build
./build/rooster
```

Same idea, one extra flag

Clang isn't the default compiler CMake picks inside this image, so it has to be named explicitly:

```
cmake -B build -G Ninja -DCMAKE_CXX_COMPILER=clang++ .
cmake --build build
./build/rooster
```

The `docker run` command mounting the folder is otherwise identical, just swap `masterclass-gcc:16` for `masterclass-clang:21`.

WHAT "MOUNTING" MEANS

Your files, not a copy

`-v "...:/workspace"` maps a folder on your machine directly into the container - editing `main.cpp` on your host and re-running `cmake --build build` inside the container rebuilds your real changes. Nothing is copied in or out by hand.

THIS IS THE SOURCE OF TRUTH

Why this course leans on Docker so heavily

Local toolchains can have quiet gaps (a locally installed compiler that compiles `std::print` but fails to *link* it, for example). Whenever something behaves unexpectedly on your machine, these two images are the reference: if it builds and runs here, the lecture's code is correct.

WHAT'S NEXT

A container with a real editor attached

So far, working in a container means an interactive shell and typing commands by hand. Lecture 2.8 connects **VS Code** to this same container - full editing, IntelliSense, and breakpoint debugging, without leaving Linux or your compiler of choice.